

Guía: Planes lógicos y físicos

Algoritmos de join, selección y ordenamiento de operaciones

Formato. En cada ejercicio se entrega un esquema con estadísticas (páginas P y tuplas T por tabla, selectividad de los filtros), los índices disponibles y el tamaño del buffer B . Para cada consulta se pide:

- Un **plan lógico** *left-deep* (es decir, el operando derecho de cada join es siempre una relación base o el resultado de una selección sobre una relación base). Dibuja el árbol y justifica el orden elegido en términos del tamaño esperado del resultado intermedio y de cómo se aprovechan los filtros.
- Un **plan físico**, indicando el algoritmo escogido para cada operación (selección, join). Justifica cada elección comparando, al menos cualitativamente, el costo de las alternativas razonables.

Ejercicio 1. Push-down de selección y join in-memory Esquema:

- Cliente(cid, nombre, segmento, ciudad): $P = 400, T = 20,000$.
- Pedido(pid, cid, fecha, monto): $P = 5,000, T = 250,000$.
- Detalle(pid, articulo, cantidad): $P = 8,000, T = 800,000$.

Índices: ninguno. **Buffer:** $B = 102$ páginas.

Selectividades y cardinalidades estimadas:

- $\sigma_{\text{segmento}='VIP'}(\text{Cliente})$ retiene el 1 % de las tuplas: ≈ 200 tuplas, ≈ 4 páginas.
- Cada cliente VIP tiene en promedio 25 pedidos, por lo que $\sigma(\text{Cliente}) \bowtie \text{Pedido}$ produce $\approx 5,000$ tuplas que ocupan ≈ 100 páginas materializadas.
- Cada pedido tiene en promedio 4 ítems en Detalle.

Consulta:

```
SELECT C.nombre, D.articulo, D.cantidad
FROM   Cliente C
JOIN   Pedido  P ON C.cid = P.cid
JOIN   Detalle D ON P.pid = D.pid
WHERE  C.segmento = 'VIP';
```

Sugerencia para la justificación: compara qué pasa con el plan “ingenuo” que primero hace $\text{Cliente} \bowtie \text{Pedido}$ sin empujar la selección, y luego aplica el filtro al final.

Ejercicio 2. Selección muy selectiva más índices Esquema:

- Sucursal(suid, nombre, region): $P = 50, T = 1,000$.
- Empleado(eid, suid, nombre, cargo): $P = 600, T = 30,000$.
- Sueldo(eid, anio, monto): $P = 4,000, T = 200,000$.

Índices:

- B+Tree *no clustered* en Empleado(suid), altura $h = 2$.
- B+Tree *clustered* en Sueldo(eid), altura $h = 3$.

Buffer: $B = 22$ páginas.

Selectividades y cardinalidades estimadas:

- $\sigma_{\text{region}='XV'}(\text{Sucursal})$ retiene el 0,5 %: ≈ 5 tuplas, 1 página.
- Cada sucursal tiene en promedio 30 empleados, por lo que el primer join produce ≈ 150 tuplas (≈ 3 páginas).
- Cada empleado tiene 5 registros de sueldo (uno por año), por lo que el resultado final tiene ≈ 750 tuplas.

Consulta:

```
SELECT  S.nombre, E.nombre, Su.anio, Su.monto
FROM    Sucursal S
JOIN    Empleado E  ON S.suid = E.suid
JOIN    Sueldo      Su ON E.eid = Su.eid
WHERE   S.region = 'XV';
```

Sugerencia para la justificación: compare INLJ contra Hash Join in-memory para cada uno de los dos joins. ¿Cómo afecta que el índice de Empleado sea *no clustered* y el de Sueldo sea *clustered*?

Join y orden Esquema:

- Pelicula(pid, titulo, anio, genero): $P = 800, T = 40,000$.
- Reparto(pid, aid, rol): $P = 2,000, T = 200,000$.
- Actor(aid, nombre, nacionalidad): $P = 500, T = 25,000$.

Índices:

- B+Tree *clustered* en Pelicula(pid), altura $h = 3$.
- B+Tree *clustered* en Reparto(pid), altura $h = 4$.

Buffer: $B = 12$ páginas.

Selectividades y cardinalidades estimadas:

- $\sigma_{\text{anio} \geq 2020}(\text{Pelicula})$ retiene el 30 %: $\approx 12,000$ tuplas, ≈ 240 páginas.
- Cada película tiene en promedio 5 actores, por lo que $\sigma(\text{Pelicula}) \bowtie \text{Reparto}$ produce $\approx 60,000$ tuplas (≈ 600 páginas).

Consulta:

```
SELECT  P.titulo, A.nombre, R.rol
FROM    Pelicula P
JOIN    Reparto  R ON P.pid = R.pid
JOIN    Actor    A ON R.aid = A.aid
WHERE   P.anio >= 2020;
```

Sugerencia para la justificación: notar que con $B = 12$ ningún operando del primer join cabe en el buffer. ¿Qué join podemos usar dados los índices que tenemos?. Para el segundo join, ¿podemos repetir la estrategia?

Ejercicio 4. El orden de los joins importa Esquema (cadena Universidad – Curso – Inscripcion):

- Universidad(uid, nombre, pais): $P = 5, T = 200$.
- Curso(cid, uid, departamento): $P = 200, T = 10,000$.
- Inscripcion(eid, cid, nota): $P = 6,000, T = 500,000$.

Índices: ninguno. **Buffer:** $B = 22$ páginas (es decir, $B - 2 = 20$).

Selectividades y cardinalidades estimadas:

- $\sigma_{\text{pais}='Chile'}(\text{Universidad})$ retiene ≈ 5 tuplas (1 página).
- $\sigma_{\text{nota} \geq 6.0}(\text{Inscripcion})$ retiene el 30 %: $\approx 150,000$ tuplas, $\approx 1,800$ páginas.
- Cada universidad chilena tiene en promedio 50 cursos, por lo que $\sigma(\text{Universidad}) \bowtie \text{Curso}$ produce ≈ 250 tuplas (≈ 5 páginas).
- Cada curso tiene en promedio 50 inscripciones; el join completo (sin el filtro sobre Inscripcion) entre cursos chilenos e inscripciones produciría $\approx 12,500$ tuplas.

Consulta:

```
SELECT  U.nombre, C.departamento, I.nota
FROM    Inscripcion I
JOIN    Curso      C ON I.cid = C.cid
JOIN    Universidad U ON C.uid = U.uid
WHERE   U.pais = 'Chile'
AND     I.nota >= 6.0;
```

Sugerencia para la justificación: hay dos órdenes left-deep válidos (después de empujar las selecciones). Comparar el tamaño del resultado intermedio en cada orden y discuta cuál algoritmo de join queda viable en cada caso (en particular: ¿cabe alguno de los operandos en el buffer? ¿conviene Hash Join in-memory, Grace Hash Join, BNLJ, o SMJ?).